Checking Concurrent Operational Procedures by Bounded Exploration of a Typed Runbook Model

Preprint. Not peer reviewed.

October 2026

Abstract

Many outages begin with an operator procedure: a maintenance command that removes too much capacity, a rollback across an incompatible migration, a restore that runs while applications still write. We present frv, a bounded model checker for operational runbooks. A runbook is written as a small typed model (system state, steps with dependencies, preconditions) and frv explores dependency-respecting step orders, checking preconditions and post-state invariants. It returns a minimized counterexample for a detected hazard, a safe verdict after complete exploration, or an inconclusive verdict when a search budget is reached. The contribution is deterministic, offline checking of modeled operational procedures with explicit completion status. Reanalysis of archived executions on 50 synthetic concurrent-maintenance procedures with an independent graph-theoretic oracle yields 44 correct decisive verdicts and 6 inconclusive results; a per-step check agrees on 25, and three LLM reviewers agree on 46, 50 and 26. On real incidents, we screened 39 first-party postmortems of procedure-caused outages; the 12 expressible in frv's vocabulary were modeled as pairs of the procedure as performed and as remediated. frv reports the documented hazard in all 12 as-performed procedures and accepts 10 of the remediated ones, a per-step check finds 5, text linters separate none, and strong LLM reviewers separate up to 11 pairs. The corpus, models, prompts, raw LLM responses and a blinded human annotation packet are released.

1 Introduction

Postmortems published by cloud operators repeatedly trace outages to procedures rather than to code: a capacity-removal command with a mistyped argument removed the servers behind Amazon S3's index subsystem; a fleet reboot meant for a few new servers targeted a whole data center; credentials on all four servers of a redundant set were changed at once instead of one at a time. Studies of outage reports find that operational and maintenance actions are a recurring trigger [8, 15, 6]. The procedures behind such incidents are short, and each step looks reasonable in isolation. The hazard appears only when the state left by earlier steps, or by steps other operators run at the same time, is taken into account.

Runbooks are reviewed by people, and increasingly by language models [1], but they are rarely checked. Text-level tools can flag dangerous commands (`rm -rf`, an unscoped `kubectl delete`), and infrastructure-as-code linters find smells in scripts [17], but neither reasons about how many replicas remain after the third of five parallel drains. General-purpose model checkers such as TLA+ and Alloy can [14, 12, 16], at the price of writing a specification from scratch for each procedure.

frv sits between the two. It provides a fixed vocabulary of 36 operational actions (drain, restore, scale, failover, rollback, migrate, suppress an alert, replay a queue, flush a cache, restore a snapshot, shift traffic, change DNS, rotate a credential, and others), each with a state-transformer semantics, together with built-in invariants over the resulting state (minimum available replicas, quorum before a lossy failover, no rollback across an incompatible migration, bounded alert suppression, write freeze before restore, and so on). An operator writes the procedure in this vocabulary; frv explores all step orders that respect the stated dependencies and reports each violated invariant with a minimized trace.

Claim. Stateful exploration detects interleaving hazards missed by per-step checking and reports completion status separately from counterexample discovery. Safety acceptance requires complete exploration under the implemented post-action property semantics. Built-in invariants are not initial-state admission checks. On the incident translations in the evaluation, the checker reproduces each documented hazard.

Contributions.

- The frv tool: a typed runbook model, its small-step semantics, built-in safety invariants with assume/guarantee and rely/guarantee extensions, and a breadth-first explorer with partial-order reduction and counterexample minimization (Sections 3–4).

```
{"system": {"services": {"control-redis": {"min_available": 3, "replicas": [
    {"id": "redis-1", "region": "us"}, ..., {"id": "redis-4", "region": "us"}]}}},
 "allow_reordering": true,
 "steps": [
  {"id": "rotate-redis-1", "action": "drain_replica",
   "params": {"service": "control-redis", "replica": "redis-1"}},
  {"id": "reconfigure-clients-redis-1", "action": "restore_replica",
   "after": ["rotate-redis-1"], "params": {...}},
  {"id": "rotate-redis-2", "action": "drain_replica",
   "after": ["reconfigure-clients-redis-1"], "params": {...}}, ...]}
```

Figure 1: Excerpt of the remediated model of the Heroku 2014 credential rotation: each server is taken out of use, and clients are reconfigured, before the next one is touched.

- An evaluation set grounded in external sources: 42 first-party postmortems screened with recorded verification, 12 modeled as as-performed/remediated pairs with the hazard label taken from the postmortem, and a pinned corpus of 1358 public runbook files from six open-source repositories (Section 5.2).
- A comparison against a document-order simulation, a per-step checker, frv's own prose linter, ShellCheck, ansible-lint and three LLM reviewers with recorded prompts and responses (Section 5).

2 Background and Related Work

Outage studies. Gunawi et al. analyse hundreds of public outage reports and list maintenance and configuration actions among the root causes [8]; Liu et al. and Ghosh et al. study production incidents at a large cloud provider [15, 6]. These studies motivate our incident set but do not check procedures.

Configuration and infrastructure-as-code analysis. Configuration-error research validates values against constraints [18]; IaC smell detectors find insecure patterns in Ansible, Chef and Puppet scripts [17]. ShellCheck [10] and ansible-lint [2] are the linters operators actually run on runbook snippets and playbooks. All of these inspect one artefact at a time and do not track system state across steps.

Model checking. Explicit-state checkers such as SPIN [11] and bounded model checking [3] explore interleavings of concurrent processes; partial-order reduction prunes orders of independent steps [7]. TLA+ is used at Amazon to check distributed protocols [16, 14]. frv applies the same explicit-state idea to a domain-specific vocabulary, so that the specification is the runbook itself and the invariants are shared across runbooks. It exports TLA+ and Alloy starter models but does not depend on them.

LLMs for operations. LLMs have been used to recommend root causes and mitigations for cloud incidents [1]. We use LLMs as reviewers of procedures, the role they would take in a pre-execution review.

3 Approach

Model. A runbook model has a system state and a list of steps. The state contains regions (with health), services (replicas with region, health and a drained flag, and a `min_available`), databases (primary region, healthy regions, quorum and migration flags), queues, caches, object buckets (snapshot, RPO/RTO, write freeze), alerts, feature flags, deployments, traffic routes, DNS records and credentials. A step names one of 36 actions with typed parameters, an optional set of predecessor steps (`after`), and optional preconditions (`requires`) and promised effects (`effects`) drawn from 37 condition kinds. Figure 1 shows a model.

Semantics. Each action denotes a partial state transformer that may raise an *undefined-action* error (unknown entity, restoring a replica that is not drained). A step is *enabled* in a configuration (s, D), where D is the set of completed steps, when all of its predecessors are in D. Executing an enabled step t checks its preconditions and the action guards in s (for example, a lossy failover requires confirmed quorum, a snapshot restore requires frozen writes), applies the action to obtain s', and checks the post-state invariants in s' (for example, every service has at least `min_available` available replicas and not all replicas drained; no traffic goes to a region without capacity; a DNS record does not answer from two regions inside the TTL window). Preconditions and effects follow the usual Hoare-style reading [9]. Assume/guarantee contracts

add per-runbook state predicates checked in every reached state; rely/guarantee entries describe an external actor whose action may run before a step and whose interference must preserve stated conditions [13].

Exploration. frv performs breadth-first search over configurations (s, D) up to a depth bound, deduplicating configurations by a state fingerprint that preserves replica list order, because downsizing keeps a list prefix. For runbooks consisting entirely of pairwise disjoint restart, flag-toggle and alert-suppression steps without dependencies, precondition/effect annotations or runbook contracts, only one step is expanded. Other actions retain every enabled branch. This is a restricted form of partial-order reduction [7]. Every violated property is reported once with the first trace that reached it, and the trace is minimized by greedily dropping steps whose removal keeps the violation reachable and the dependencies satisfied. A counterexample remains decisive when a budget is reached. An absence of violations is accepted as safe only after complete exploration; depth, state and wall-clock cutoffs, and a blocked procedure with outstanding steps under the selected scheduling policy, produce an inconclusive result. The optional dominance abstraction also reports inconclusive when it merges distinct concrete states.

4 Implementation

frv is about 10.7k lines of Python using only the standard library. Input is JSON, YAML (with PyYAML installed) or a Markdown file containing one fenced `runbook-json` block, so a model can live next to the prose runbook it encodes, with findings mapped to the source line of the offending field. The command-line tool provides `check`, `validate` (schema and reference checks without exploration), `explain` (counterexample with synthesized precondition and JSON-patch suggestions), `diff` (semantic difference between two runbook versions), `export` (TLA+ and Alloy starter models), and a prose linter, `lint-markdown`, that flags hazardous phrases in runbooks that have no executable model. Output formats include JSON, Markdown, SARIF and JUnit for CI integration. The test suite has 129 tests.

5 Evaluation

5.1 Research questions

RQ1 Does frv separate the procedure as performed in a real procedure-caused incident from the remediated procedure, and how do simpler checkers, linters and LLM reviewers compare?

RQ2 How do frv, a per-step checker and LLM reviewers compare on concurrent procedures where the hazard depends on interleaving, as the procedures grow?

RQ3 How often do public runbooks contain steps in frv's hazard classes, and how many of those are expressible?

5.2 Setup

Incident set (RQ1). We collected first-party postmortems in which an operator-executed procedure, maintenance playbook or operations script caused or materially worsened an outage. Each candidate was fetched and checked against the first-party page or an archived copy, with one quotation verified verbatim. Of 42 candidates, 39 met the criteria. Before modeling, the files that define frv's actions, conditions and invariants were frozen (their SHA-256 hashes are checked by the evaluation scripts), and a translation guide fixed how procedure text becomes a model: steps only for state-changing actions, dependencies only where the text orders steps, preconditions only where the text tells the operator to check, and quantities only as stated, with the smallest consistent configuration and a recorded assumption otherwise. A case whose hazard needs a concept outside the vocabulary is recorded as not expressible and not approximated. 12 incidents were expressible; the 27 others need concepts such as wrong-target identifiers, BGP route attraction, capacity quantities other than replica counts, or the binding between a consumer and the credential it holds. For each modeled incident we wrote two models, the procedure as performed and the procedure with only the safeguards the postmortem announces, and a prose rendering of each (system facts and numbered steps, no outcome) for the text-only methods. The hazard label of the as-performed variant comes from the postmortem; the remediated variant is expected to be safe.

Table 1: Pairs separated per incident: the method flags the as-performed procedure with the documented hazard and accepts the remediated one. LLM columns use the temperature-0 run.

Incident	frv	Doc. order	Per step	Prose lint	Sonnet 5.5	Gem. 3.8 Fl.	GPT-5.4 mini
aws-ec2-dns-seoul-2018-11-22	✓	✓	–	–	✓	✓	✓
aws-s3-2017-02-28	✓	✓	–	–	✓	✓	–
circleci-2021-11-08	✓	✓	–	–	✓	✓	–
dropbox-2014-01-10	–	–	–	–	✓	✓	✓
gcp-networking-2019-06-02	✓	✓	–	–	✓	✓	✓
github-2018-10-21	✓	✓	✓	–	✓	✓	✓
heroku-redis-credentials-2014-06-23	✓	✓	–	–	✓	✓	✓
joyent-us-east-1-2014-05-27	–	–	✓	–	✓	✓	✓
mezmo-2022-04-05	✓	✓	✓	–	✓	✓	–
reddit-2016-08-11	✓	✓	✓	–	–	–	–
travis-ci-2018-03-13	✓	✓	–	–	✓	✓	✓
valtown-2025-04-02	✓	✓	–	–	✓	✓	–
Total	10	10	4	0	11	11	7

Table 2: Concurrent maintenance (synthetic): decisive verdicts agreeing with the width oracle, by number of operators P (10 instances each, 5 safe and 5 unsafe), with frv's largest state count and run time.

P	frv	Doc. order	Per step	Sonnet 5.5	Gem. 3.8 Fl.	GPT-5.4 mini	states	s
3	10/10	5/10	5/10	9/10	10/10	7/10	64	0.02
5	10/10	5/10	5/10	8/10	10/10	5/10	1024	0.43
8	10/10	5/10	5/10	9/10	10/10	5/10	65536	28.68
12	9/10	5/10	5/10	10/10	10/10	4/10	1191313	917.70
16	5/10	5/10	5/10	10/10	10/10	5/10	1245450	1027.77

Concurrent maintenance (RQ2). This set is synthetic. Each instance has $P \in \{3, 5, 8, 12, 16\}$ operators; operator i drains replica i of a service, patches it and restores it, and random cross-dependencies state that one operator starts only after another has restored its replica. The procedure is unsafe exactly when more operators can be drained at the same time than the service's slack (replicas minus minimum). That maximum is the width of the operators' precedence order, which we compute independently of frv with Dilworth's theorem [5] (operators minus a maximum matching in the comparability graph). For each P we generated 5 safe instances (slack equal to the width) and 5 unsafe ones (slack one less), 50 in total.

Corpus (RQ3). We pinned six public runbook sources to commits: GitLab's infrastructure runbooks (MIT), OpenShift alert runbooks (Apache-2.0), the prometheus-operator runbooks (Apache-2.0), the Grafana Mimir runbook index (AGPL-3.0, referenced by hash and not redistributed), the etcd v3.5 operations guide (CC-BY-4.0) and kubespray's operations documents and playbooks (Apache-2.0), 1358 Markdown files in total. From the files where the prose linter reports one of seven state-dependent hazard rules we drew a seeded sample of three files per rule (21 files) before reading any of them.

Baselines. *Document order* runs the same semantics on the single order in which steps are written. *Per step* evaluates each step's preconditions, guards and post-state invariants against the initial state, which is what a rule-per-step checker sees. *Prose lint* is frv's own phrase-level linter on the prose rendering. ShellCheck 0.11.0 and ansible-lint 26.9.0 were run on the corpus. The LLM reviewers are Claude Sonnet 5.5, Gemini 3.8 Flash and GPT-5.4 mini via OpenRouter, given a fixed system prompt asking whether executing the procedure as written can cause an outage or data loss, with a list of hazard categories; each prose rendering is reviewed in a separate request, once at temperature 0 and three times at temperature 1. All prompts and responses are stored.

Translation. The incident and corpus models were produced by an LLM-based coding agent following the translation guide. Every model records its assumptions, and a human review packet (model faithfulness and finding validity) is included with the artefact.

5.3 RQ1: Real procedure-caused incidents

Table 1 reports, per incident, whether each method flags the as-performed procedure with the documented hazard and accepts the remediated one. frv flags the documented hazard in all 12 as-performed procedures and accepts 10 of the 12 remediated procedures. The two remediated procedures it rejects (Joyent, Dropbox) drain every machine of a pool whose minimum is zero; the invariant that no service may have all replicas drained fires on such a pool. A waiver, which frv supports, would record that exception; we report the result without one. Each check took at most 185 ms.

The document-order simulation gives the same verdicts as frv on every incident: in these procedures the hazardous state is reached in the written order. The per-step check finds the hazard in only 5 of 12 as-performed procedures, because in the others no single step is unsafe from the initial state; the hazard is the cumulative effect of several removals (S3, GCP, Heroku, Dropbox), or a rollback that is unsafe only after the migration step (CircleCI, Val Town). It also rejects a remediated restore whose write-freeze precondition is established by the step before it. The prose linter separates no pair: its phrase rules fire on both variants or on neither. On the corpus, ShellCheck reports 2584 findings in 947 of 2328 shell blocks and ansible-lint one syntax finding on the kubespray playbooks; their rules concern quoting, syntax and module use, not capacity, quorum or ordering.

The strong LLM reviewers do well on these short procedures. Claude Sonnet 5.5 and Gemini 3.8 Flash each separate 11 pairs; both reject the remediated Reddit procedure, in which the autoscaler may still remove servers one at a time while configuration management re-enables it, a judgement a reviewer could defend. GPT-5.4 mini separates 7 pairs, rejecting 5 remediated procedures. Verdicts were stable across the four runs for Sonnet (24/24 cells) and Flash (24/24). Several of these incidents are well known, so the reviewers may have seen discussions of them.

5.4 RQ2: Concurrent maintenance

Table 2 reports decisive agreement with the width oracle. Reanalysis of the saved runs yields 44 correct decisive results and 6 inconclusive results among 50 instances, with at most 1245450 explored states and 1027.8 s per instance. The exploration budget is 900 seconds; recorded wall times include counterexample minimization. The document-order simulation agrees on 25: the written order runs operators one after another and never reaches the concurrent state, so it misses 25 unsafe instances. The per-step check agrees on 25, missing 25 unsafe instances, because a single drain from the initial state never exceeds the slack. The LLM reviewers agree on 46 (Sonnet 5.5), 50 (Gemini 3.8 Flash) and 26 (GPT-5.4 mini) of 50 instances.

5.5 RQ3: Applicability to public runbooks

Of the 21 sampled files, 13 contain an operator step in the class the linter flagged; the others matched on headings, background prose or unrelated uses of words such as “purge”. 6 of the 13 are expressible: three alert silences, two node drains and one dead-letter replay. With the documented two-hour default of Alertmanager’s silence form, the silences are within frv’s default bound; the drains are safe in the smallest configuration consistent with the text; the replay is flagged because the runbook states no deduplication or idempotency for replayed items, an obligation the runbook leaves to the reader. The remaining steps need concepts the vocabulary lacks, most often the binding between a consumer and the credential it uses.

The linter that selects candidate files is a triage aid rather than a detector. On a blinded sample of 137 findings, LLM-assisted labels judge 44 % (Sonnet 5.5) and 49 % (GPT-5.4 mini) to be operator steps of the flagged class (agreement between the two labelers $\kappa = 0.63$ [4]), and 4 of 80 randomly drawn unflagged lines are judged to be such steps. These are not human labels; the blinded packet and agreement script for two human annotators are part of the artefact.

5.6 Independent semantic-boundary controls

A prospectively frozen local experiment used a separate plain-list transition implementation and enumerated every dependency-respecting permutation of three actions: regional scale-up, prefix-preserving scale-down, and traffic shift. It swept eight acyclic dependency subsets, two initial replica orders, two expansion regions, and two traffic targets. The 64 synthetic models produced 512 comparisons across four exploration strategies and reduction on/off. All decisive verdicts agreed with the independent oracle before and after the following repairs. This covers a finite indirect-dependency boundary; it is neither an external TLC/SPIN comparison nor a proof for the full action vocabulary.

A separate state-pair diagnostic found that the old fingerprint equated two replica lists with the same members but different order, although downsizing them retained replicas in different regions. The fingerprint now preserves list order. This falsifies the old abstraction’s transition congruence, but we did not demonstrate

a reachable merged configuration yielding a false safe result. A forward dependency under document-order scheduling exposed another boundary: the checker previously returned safe after exploring one state while no step was enabled and all work remained outstanding. It now reports inconclusive. Enabling dependency-respecting reordering makes that same procedure executable. Complete raw inputs, before/after outcomes, and the initially incorrect state-pair harness (which had also changed replica identifiers) are retained; the corrected harness preserves identifiers. The earlier 512 independent flag/precondition controls and eight future-enabled attack variants were replayed after the repair.

6 Discussion

Where exploration matters. On the real incidents the hazardous state was reachable in the written order, so a straight-line simulation over the same semantics already separates the pairs; what it needs, and the per-step check and the text tools lack, is state threaded from step to step. Exploration becomes necessary when steps are concurrent. Maintenance windows with several operators, automation running beside a human (the Reddit autoscaler) and rolling procedures whose batches overlap all have this shape, and RQ2 shows that both the written order and per-step checking then miss unsafe procedures that frv finds.

Relation to LLM review. On short procedures with explicit system facts, strong LLM reviewers separate most incident pairs and their verdicts are stable. A completed frv search reports safety under the implemented model semantics, costs no API call, runs offline in CI, and comes with a minimized trace that names the steps and the violated invariant. The two are complementary: a reviewer can read prose that has no model, and frv can decide a model whose interleavings are too many to reason about informally.

Cost of modeling. frv requires a model. In the corpus sample, fewer than half of the hazardous steps were expressible, and in the incident set 12 of 39. The missing concepts recorded in the artefact (consumer-to-credential binding, capacity quantities, record deletion, routing state) indicate where the vocabulary would have to grow.

7 Threats to Validity

Construct. The incident models and prose renderings were written from the postmortems by an LLM-based agent; the modeler knew the outcome. The frozen vocabulary, the translation guide and the recorded assumptions limit, but do not remove, the risk that a model encodes the hazard. The human review packet is intended to address this and has not yet been completed. *Internal.* Several minimum capacities are not published and were set to the smallest configuration consistent with the report; another choice could change a verdict. Two incidents use an abstraction: a database restore is modeled with the object-bucket restore semantics, and a parser restart is modeled as an alert suppression with no end. *External.* Twelve incidents is a small set, restricted to hazards in the vocabulary, and RQ2 is synthetic with one hazard shape. The LLM results hold for the three models and the prompt we used. No matched established workflow checker, independent validation of the complete semantics, or held-out measure of operational benefit is reported. The source semantics hash freeze describes the historical incident study, not the repaired checker; current file hashes are recorded separately. *Labels.* The corpus labels in RQ3 are LLM-assisted.

8 Conclusion

frv checks operational procedures against shared safety invariants by exploring every order their dependencies allow. On the evaluated incident translations it reproduces all twelve documented hazards. The concurrent-maintenance study yields 44 correct decisive results and six inconclusive results; strong LLM reviewers also perform well on this suite. The checker adds offline execution and reproducible counterexample traces. The tool, the pinned corpus, the incident models, the prompts with all responses, and the annotation materials are available in the repository.

References

[1] Toufique Ahmed, Supriyo Ghosh, Chetan Bansal, Thomas Zimmermann, Xuchao Zhang, and Saravan Rajmohan. Recommending root-cause and mitigation steps for cloud incidents using large language

models. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*, pages 1737–1749, 2023.

[2] Ansible community. ansible-lint, version 26.9.0. `https://github.com/ansible/ansible-lint`. Accessed 2026-10-07.

[3] Armin Biere, Alessandro Cimatti, Edmund Clarke, and Yunshan Zhu. Symbolic model checking without BDDs. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, volume 1579 of *LNCS*, pages 193–207, 1999.

[4] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960.

[5] R. P. Dilworth. A decomposition theorem for partially ordered sets. *Annals of Mathematics*, 51(1):161–166, 1950.

[6] Supriyo Ghosh, Manish Shetty, Chetan Bansal, and Suman Nath. How to fight production incidents? an empirical study on a large-scale cloud service. In *Proceedings of the 13th Symposium on Cloud Computing (SoCC)*, pages 126–141, 2022.

[7] Patrice Godefroid. *Partial-Order Methods for the Verification of Concurrent Systems*, volume 1032 of *LNCS*. Springer, 1996.

[8] Haryadi S. Gunawi, Mingzhe Hao, Riza O. Suminto, Agung Laksono, Anang D. Satria, Jeffry Adityatama, and Kurnia J. Eliazar. Why does the cloud stop computing? lessons from hundreds of service outages. In *Proceedings of the Seventh ACM Symposium on Cloud Computing (SoCC)*, pages 1–16, 2016.

[9] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.

[10] Vidar Holen. ShellCheck: a static analysis tool for shell scripts, version 0.11.0. `https://www.shellcheck.net`. Accessed 2026-10-07.

[11] Gerard J. Holzmann. The model checker SPIN. *IEEE Transactions on Software Engineering*, 23(5):279–295, 1997.

[12] Daniel Jackson. Alloy: A lightweight object modelling notation. *ACM Transactions on Software Engineering and Methodology*, 11(2):256–290, 2002.

[13] Cliff B. Jones. Tentative steps toward a development method for interfering programs. *ACM Transactions on Programming Languages and Systems*, 5(4):596–619, 1983.

[14] Leslie Lamport. The temporal logic of actions. *ACM Transactions on Programming Languages and Systems*, 16(3):872–923, 1994.

[15] Haopeng Liu, Shan Lu, Madan Musuvathi, and Suman Nath. What bugs cause production cloud incidents? In *Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS)*, pages 155–162, 2019.

[16] Chris Newcombe, Tim Rath, Fan Zhang, Bogdan Munteanu, Marc Brooker, and Michael Deardeuff. How Amazon web services uses formal methods. *Communications of the ACM*, 58(4):66–73, 2015.

[17] Akond Rahman, Chris Parnin, and Laurie Williams. The seven sins: Security smells in infrastructure as code scripts. In *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, pages 164–175, 2019.

[18] Tianyin Xu and Yuanyuan Zhou. Systems approaches to tackling configuration errors: A survey. *ACM Computing Surveys*, 47(4):70:1–70:41, 2015.